

balaji-creator / Univariate-Linear-Regression

Q

+ ▾

<> Code 🔗 Pull requests ⚙️ Agents ▶ Actions 📁 Projects 📖 Wiki 🛡 Security and quality 📈 Insights

👁 Watch 0 ▾

🔗

☆ ▾

Implementation of Univariate Linear Regression

📄 BSD-3-Clause license

☆ 0 stars 🔗 2.3k forks 👁 0 watching 🔗 Branches 📈 Activity

📁 Tags

🌐 Public repository · Forked from [etjabajasphin/Univariate-Linear-Regression](#)

🔗 🔗 1 Branch 📁 0 Tags 🔗 📁

🔍 Go to file T Go to file Add file ➕ Code ⋮

This branch is **1 commit ahead of** etjabajasphin/Univariate-Linear-Regression:main .

🔗 Contribute ▾

🔄 Sync fork ▾

 balaji-creator succes	d984c75 · now	🕒
📄 566753653-72c646b4-212d...	succes	now
📄 566753706-ccc21af2-f95d-...	succes	now
📄 566754298-90267474-84ee...	succes	now
📄 LICENSE	Initial commit	5 years ago
📄 README.md	succes	now
📄 eq1.jpg	experiment added	5 years ago
📄 eq2.jpg	experiment added	5 years ago
📄 input.jpg	experiment added	5 years ago

📖 README 📄 License

✎ ☰

Implementation of Univariate Linear Regression

Aim:

To implement univariate Linear Regression to fit a straight line using least squares.

Equipment's required:

1. Hardware – PCs

2. Anaconda – Python 3.7 Installation / Moodle-Code Runner

Algorithm:

1. Get the independent variable X and dependent variable Y.
2. Calculate the mean of the X -values and the mean of the Y -values.

$$m = \frac{\sum_{i=1}^n (x_i - \bar{X})(y_i - \bar{Y})}{\sum_{i=1}^n (x_i - \bar{X})^2}$$

3. Find the slope m of the line of best fit using the formula.

$$b = \bar{Y} - m\bar{X}$$

4. Compute the y -intercept of the line by using the formula:
5. Use the slope m and the y -intercept to form the equation of the line.
6. Obtain the straight line equation $Y=mX+b$ and plot the scatterplot.

Program

```
import numpy as np
import matplotlib.pyplot as plt
x = np.array([0,1,2,3,4,5,6,7,8,9])
y = np.array([1,3,2,5,7,8,8,9,10,12])
plt.scatter(x,y)
plt.show()
xmean = np.mean(x)
ymean = np.mean(y)
num=0
den=0
for i in range(len(x)):
    num+=(x[i]-xmean)*(y[i]-ymean)
    den+=(x[i]-xmean)**2
m = num/den
b = ymean - m*xmean
print(m,b)
ypred = m*x+b
print(ypred)

plt.scatter(x,y,color='Red')
plt.plot(x,ypred,color='Blue')
plt.show()
```



Output

jupyter Untitled33 Last Checkpoint: 42 minutes ago (unsaved changes)



Logout

File Edit View Insert Cell Kernel Widgets Help

Trusted



Python 3 (ipykernel)

Run Code

```
In [9]: import numpy as np
import matplotlib.pyplot as plt
x = np.array([0,1,2,3,4,5,6,7,8,9])
y = np.array([1,3,2,5,7,8,8,9,10,12])
plt.scatter(x,y)
plt.show()
xmean = np.mean(x)
ymean = np.mean(y)
num=0
den=0
for i in range(len(x)):
    num+=(x[i]-xmean)*(y[i]-ymean)
    den+=(x[i]-xmean)**2
m = num/den
b = ymean - m*xmean
print(m,b)
ypred = m*x+b
print(ypred)

plt.scatter(x,y,color='Red')
plt.plot(x,ypred,color='Blue')
plt.show()
```

jupyter Untitled33 Last Checkpoint: 43 minutes ago (unsaved changes)



Logout

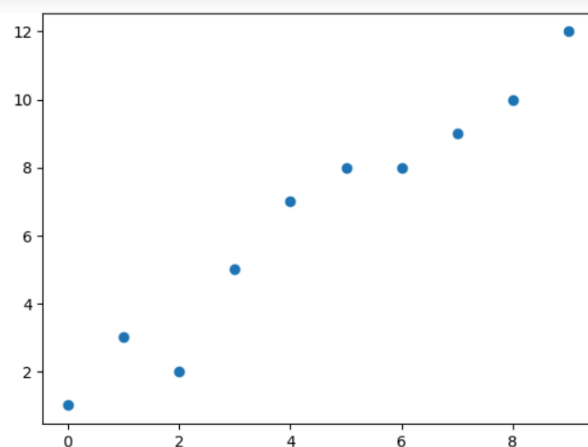
File Edit View Insert Cell Kernel Widgets Help

Trusted



Python 3 (ipykernel)

Run Code



```
1.1696969696969697 1.2363636363636363
[ 1.23636364  2.40606061  3.57575758  4.74545455  5.91515152  7.08484848
  8.25454545  9.42424242 10.59393939 11.76363636]
```

jupyter Untitled33 Last Checkpoint: an hour ago (unsaved changes)



Logout

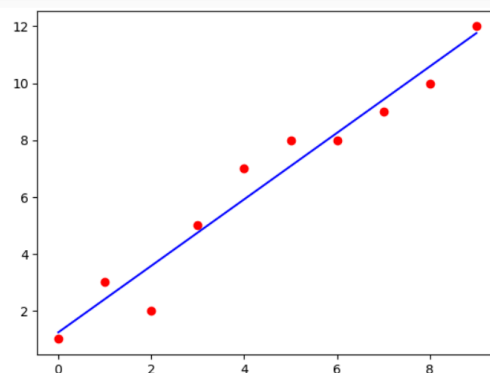
File Edit View Insert Cell Kernel Widgets Help

Trusted



Python 3 (ipykernel)

Run Code



Releases

No releases published

[Create a new release](#)

Packages

No packages published

[Publish your first package](#)

Contributors

No contributors